

NumPy & Pandas for Machine Learning

Practical Session 1 - Foundations Lab Sheet

B.Tech (Computer Science Engineering) - III Year
Machine Learning Laboratory

June 22, 2026

Instructions for Students: Work through each topic in order. Read the *Concept* box, run the *Example Code* yourself in a Jupyter Notebook / Google Colab / VS Code, and then solve the *Exercise* underneath *before* moving to the next topic. Keep all your code in a single notebook named `lab1_yourrollno.ipynb`.

Contents

Why NumPy and Pandas?	2
1 Part I — NumPy for Machine Learning	3
1.1 Creating Arrays	3
1.2 Array Attributes & Reshaping	3
1.3 Indexing, Slicing & Boolean Masking	4
1.4 Arithmetic Operations & Broadcasting	4
1.5 Aggregations & Axis-wise Statistics	5
1.6 Stacking, Concatenation & Splitting	5
1.7 Linear Algebra Essentials	6
1.8 Random Module for ML	6
1.9 Sorting, Searching & Unique Values	7
2 Part II — Pandas for Machine Learning	8
2.1 Series & DataFrame Basics	8
2.2 Loading & Exploring a Real Dataset	8
2.3 Selecting & Filtering Data	9
2.4 Handling Missing Data	9
2.5 Feature Engineering: <code>apply</code> & <code>map</code>	10
2.6 Sorting Values	11
2.7 GroupBy & Aggregation	11
2.8 Merging & Concatenating DataFrames	12
2.9 Encoding Categorical Variables	12
2.10 Correlation Analysis & Train/Test Sampling	13
3 Capstone Exercise — Putting It All Together	14
Appendix: Quick Reference Cheat Sheet	15

Why NumPy and Pandas?

Every Machine Learning pipeline begins with data, and data in Python almost always passes through two libraries first. **NumPy** gives us the `ndarray`, a fast, memory-efficient, multi-dimensional array that every ML library (scikit-learn, TensorFlow, PyTorch) is built on top of — features, weights, gradients, and predictions are all just NumPy arrays under the hood. **Pandas** gives us the `DataFrame`, the standard way to load, clean, explore, and transform tabular datasets before they are converted into NumPy arrays and fed to a model. Mastering these two libraries is therefore not optional groundwork — it *is* the first half of doing Machine Learning in Python.

A note on datasets. Pandas itself does not ship with sample datasets. The standard practice in ML courses is to use the small **toy datasets bundled with scikit-learn** (no internet required) and load them straight into a Pandas `DataFrame`. Throughout this sheet we use the **Iris** dataset for examples. For every exercise, you will practice the same operations on the **Wine** dataset (`sklearn.datasets.load_wine`) — this forces you to adapt the code rather than copy it.

Example Code

```
from sklearn.datasets import load_iris, load_wine
import pandas as pd

iris = load_iris()
iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)
iris_df['target'] = iris.target

wine = load_wine()
wine_df = pd.DataFrame(wine.data, columns=wine.feature_names)
wine_df['target'] = wine.target
print(iris_df.shape, wine_df.shape)
```

1 Part I — NumPy for Machine Learning

1.1 Creating Arrays

Concept

The `ndarray` is NumPy's core data structure. Arrays can be built from Python lists, or generated directly with helper functions such as `zeros`, `ones`, `eye`, `arange`, and `linspace`. These generators are used constantly in ML code to initialize weight matrices, bias vectors, and evaluation grids.

Example Code

```
import numpy as np

a = np.array([1, 2, 3, 4, 5]) # from a Python list

zeros = np.zeros((2, 3)) # 2x3 array of 0.0
ones = np.ones((3,)) # 1-D array of 1.0
ident = np.eye(3) # 3x3 identity matrix
rng_a = np.arange(0, 10, 2) # [0 2 4 6 8]
lin_a = np.linspace(0, 1, 5) # 5 evenly spaced points in [0, 1]

print(a, a.dtype)
print(zeros)
print(rng_a)
print(lin_a)
```

Exercise

1. Create a 4×4 identity matrix and a 3×5 array of all 7s using a single NumPy call (hint: `np.full`).
2. Create an array of 10 numbers evenly spaced between -1 and 1 using `np.linspace`.
3. Convert the Python list `[2, 4, 6, 8]` into a NumPy array and print its `dtype`.

1.2 Array Attributes & Reshaping

Concept

`shape`, `ndim`, `size`, and `dtype` describe an array's structure – in ML, `shape` is the single most-checked attribute, since a mismatched shape is the most common bug when feeding data into a model. `reshape` and `.T` (transpose) rearrange data without copying it, which is essential when converting a flat feature vector into a matrix, or aligning matrices for multiplication.

Example Code

```
import numpy as np

X = np.arange(12).reshape(3, 4) # 3x4 matrix
print("shape:", X.shape, "ndim:", X.ndim, "size:", X.size, "dtype:", X.dtype)

flat = X.ravel() # flatten to 1-D (view)
Xt = X.T # transpose → 4x3
print(flat)
print(Xt)
```

Exercise

1. Create a 1-D array with values 0 to 23 and reshape it into a 4×6 matrix, then into a $2 \times 3 \times 4$ array.
2. Take the 4×6 matrix above and transpose it. Print the shape before and after.
3. Flatten the transposed matrix back to 1-D using `ravel()` and confirm its length is 24.

1.3 Indexing, Slicing & Boolean Masking**Concept**

NumPy supports the same `[start:stop]` slicing as Python lists, but extended to multiple dimensions with a comma: `X[rows, cols]`. **Boolean masking** – indexing an array with a condition – is the backbone of data filtering in ML (e.g. “select all rows where the label equals 1”).

Example Code

```
import numpy as np
X = np.arange(12).reshape(3, 4)

print(X[1, 2]) # single element: row 1, col 2
print(X[:, 1]) # entire column 1
print(X[0:2, 1:3]) # sub-matrix (rows 0-1, cols 1-2)

mask = X > 6
print(X[mask]) # boolean masking → elements greater than 6
```

Exercise

1. From a 5×5 array of `np.arange(25).reshape(5,5)`, extract the middle 3×3 sub-matrix using slicing.
2. Extract all even numbers from that 5×5 array using a boolean mask.
3. Replace every element greater than 15 with the value -1 (hint: `X[X > 15] = -1`).

1.4 Arithmetic Operations & Broadcasting**Concept**

NumPy applies `+`, `-`, `*`, `/` **elementwise** – no loops needed. **Broadcasting** is the rule that lets NumPy operate on arrays of different (but compatible) shapes, e.g. adding a single row vector to every row of a matrix. This is exactly how a bias term is added to every sample in a batch during training.

Example Code

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b) # elementwise addition
print(a * 2) # scalar broadcast
M = np.ones((3, 3))
print(M + a) # row vector broadcast over every row of a 3x3
```

Exercise

1. Given `prices = np.array([100, 250, 80, 500])` and a flat discount of 10, compute the discounted prices using broadcasting (no loop).
2. Create a 4×3 matrix of ones and add the row vector `[1, 2, 3]` to it using broadcasting. Predict the output shape before running the code.
3. Normalize the array `[2, 4, 4, 4, 5, 5, 7, 9]` by subtracting its mean and dividing by its standard deviation (this is exactly how features are standardized before training a model).

1.5 Aggregations & Axis-wise Statistics

Concept

Functions like `sum`, `mean`, `std`, `min`, `max`, `argmin`, and `argmax` reduce an array to summary statistics. The `axis` parameter controls the direction of reduction: `axis=0` collapses rows (column-wise stats), `axis=1` collapses columns (row-wise stats). This is exactly how you compute per-feature mean and standard deviation across an entire dataset.

Example Code

```
import numpy as np
X = np.array([[1, 2, 3], [4, 5, 6]])

print(X.sum()) # total sum
print(X.sum(axis=0)) # column-wise sum
print(X.mean(axis=1)) # row-wise mean
print(X.std()) # overall standard deviation
print(X.argmax(axis=1)) # index of the max value in each row
```

Exercise

1. For a 5×4 random integer matrix (`np.random.randint(0,100,(5,4))`), compute the mean of every *column* and the maximum of every *row*.
2. Find the index of the overall minimum value in the matrix using `argmin` (hint: you may need `np.unravel_index`).
3. Using only NumPy aggregation functions, compute the range (`max - min`) of each column.

1.6 Stacking, Concatenation & Splitting

Concept

ML pipelines frequently need to combine arrays – stacking feature columns together, merging batches, or splitting a dataset into folds. `vstack/hstack` stack arrays vertically/horizontally, `concatenate` is the general-purpose version, and `split` divides an array into equal pieces.

Example Code

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.vstack([a, b])) # stack as rows → 2x3
print(np.hstack([a, b])) # stack as one row → 1x6
print(np.concatenate([a, b])) # same as hstack for 1-D arrays
```

```
left, right = np.split(np.arange(8), 2)
print(left, right)
```

Exercise

1. Create two 2×2 matrices and combine them into a 4×2 matrix using `vstack`, then into a 2×4 matrix using `hstack`.
2. Split an array of 12 elements into 3 equal parts using `np.split` and print each part.
3. Use `np.column_stack` to combine three separate 1-D feature arrays `age`, `height`, `weight` (each of length 5) into a single 5×3 feature matrix.

1.7 Linear Algebra Essentials

Concept

Linear algebra is the mathematical language of ML: `dot` / `@` compute matrix products (used in every forward pass of a neural network and in linear regression predictions $\hat{y} = Xw$), `inv` computes a matrix inverse (used in the closed-form Normal Equation for linear regression), and `norm` computes vector length (used in regularization and distance-based algorithms like k-NN).

Example Code

```
import numpy as np
X = np.array([[1., 2.], [3., 4.]])
w = np.array([0.5, 0.5])

print(X.dot(w)) # matrix-vector product
print(X @ X) # matrix-matrix product
print(np.linalg.inv(X)) # matrix inverse
print(np.linalg.det(X)) # determinant
print(np.linalg.norm(w)) # Euclidean (L2) norm
```

Exercise

1. Given feature matrix `X` of shape $(4, 2)$ and weight vector `w` of shape $(2,)$, compute the predictions `X @ w` for `X = [[1, 1], [1, 2], [1, 3], [1, 4]]` and `w = [2, 3]`.
2. Compute the L1 norm (`ord=1`) and L2 norm (`ord=2`, the default) of the vector `[3, -4, 0, 12]` and compare them.
3. Verify that $XX^{-1} = I$ for a 3×3 invertible matrix of your choice, using `np.linalg.inv` and `np.allclose`.

1.8 Random Module for ML

Concept

`np.random` is used to initialize model weights, shuffle datasets, and draw random samples for train/test splits. `seed` makes results **reproducible** – always set it before sharing or grading random code. `rand` draws uniform numbers, `randn` draws standard normal numbers, and `choice` / `shuffle` are used for sampling and shuffling.

Example Code

```
import numpy as np
np.random.seed(42) # reproducibility

print(np.random.rand(3)) # uniform numbers in [0, 1)
print(np.random.randn(3)) # standard normal numbers
print(np.random.randint(0, 10, 5)) # 5 random integers in [0, 10)

idx = np.arange(6)
np.random.shuffle(idx) # shuffle in place
print(idx)
print(np.random.choice([10, 20, 30], size=2, replace=False))
```

Exercise

1. Set the seed to 7 and generate a 3×3 matrix of random integers between 1 and 100. Run it twice and confirm both outputs are identical.
2. Simulate initializing the weights of a tiny neural network layer: generate a 4×2 weight matrix from a standard normal distribution using `np.random.randn`.
3. Given an array `np.arange(20)`, randomly shuffle it and split it into an 80% / 20% train/test split (first 16 indices = train, last 4 = test) *after* shuffling.

1.9 Sorting, Searching & Unique Values**Concept**

`sort` returns a sorted copy while `argsort` returns the **indices** that would sort the array (very useful for ranking predictions by confidence score). `np.where` applies a condition elementwise, `unique` finds distinct values (e.g. distinct class labels), and `clip` bounds values to a range (used to cap gradients or pixel values).

Example Code

```
import numpy as np
a = np.array([3, 1, 4, 1, 5, 9, 2, 6])

print(np.sort(a)) # sorted copy
print(np.argsort(a)) # indices that would sort a
print(np.unique(a)) # unique values
print(np.where(a > 4, 1, 0)) # 1 where condition is True, else 0
print(np.clip(a, 2, 6)) # clip values to the range [2, 6]
```

Exercise

1. Given prediction confidence scores `scores = np.array([0.2, 0.91, 0.4, 0.88, 0.05])`, find the indices that would sort them in *descending* order (hint: sort `-scores`, or use slicing `::-1` on `argsort`).
2. Find how many **unique** class labels exist in `labels = np.array([0,1,2,1,0,2,2,1,0])` and how many times each one appears (hint: `return_counts=True`).
3. Use `np.where` to convert all probabilities in `p = np.array([0.3, 0.7, 0.45, 0.9, 0.1])` into binary class predictions using a threshold of 0.5.

2 Part II — Pandas for Machine Learning

2.1 Series & DataFrame Basics

Concept

A **Series** is a labeled 1-D array (think: one column), and a **DataFrame** is a labeled 2-D table (think: a spreadsheet) – a collection of Series sharing the same index. Almost every dataset you load for ML becomes a DataFrame: rows are samples, columns are features.

Example Code

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s)

df = pd.DataFrame({
    'age': [25, 32, 47, 51],
    'salary': [40000, 55000, 80000, 95000],
    'city': ['Pune', 'Mumbai', 'Pune', 'Delhi']
})
print(df)
```

Exercise

1. Create a **Series** of 5 exam scores with custom index labels 's1' to 's5', then print the score for 's3'.
2. Create a **DataFrame** with columns **name**, **marks**, and **passed** (boolean) for 4 students of your choice.
3. From the **DataFrame** above, print just the **marks** column, and separately print its data type using **.dtype**.

2.2 Loading & Exploring a Real Dataset

Concept

Once scikit-learn loads a dataset, wrap it in `pd.DataFrame` to explore it with Pandas. **head/tail** preview rows, **shape** gives (rows, columns), **info** shows column types and missing values, and **describe** gives summary statistics – **always** run these four commands immediately after loading any new dataset.

Example Code

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

print(df.head())
print(df.shape)
print(df.info())
print(df.describe())
```

Exercise

1. Load the Wine dataset (`load_wine`) into a DataFrame, adding the `target` column. Print its shape and the first 5 rows.
2. Run `.info()` on the Wine DataFrame. How many columns are there, and are there any missing values?
3. Run `.describe()` and identify which feature has the largest range ($\text{max} - \text{min}$).

2.3 Selecting & Filtering Data

Concept

Select a single column with `df['col']` or several with `df[['c1', 'c2']]`. `loc` selects by **label**, `iloc` selects by **integer position**. Boolean filtering – `df[df['col'] > x]` – is the Pandas equivalent of NumPy boolean masking, and is how you isolate a class, remove outliers, or split data by a condition.

Example Code

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

print(df['sepal length (cm)'].head(3)) # one column
print(df[['sepal length (cm)', 'target']].head(3)) # multiple columns

print(df.loc[0:2, 'sepal width (cm)']) # label-based
print(df.iloc[0:3, 1]) # position-based

big_petal = df[df['petal length (cm)'] > 6]
print(big_petal.shape)
```

Exercise

1. On the Wine DataFrame, select only the `alcohol` and `color_intensity` columns for the first 10 rows.
2. Use `loc` to select rows 5 to 10 (inclusive) and only the `magnesium` column.
3. Filter the Wine DataFrame to keep only rows where `alcohol` is greater than 14 **and** `target` equals 0 (hint: combine conditions with `&` and wrap each condition in parentheses).

2.4 Handling Missing Data

Concept

Real-world datasets almost always have missing values. `isnull()` flags them, `.sum()` counts them per column, `dropna()` removes rows/columns containing them, and `fillna()` imputes a replacement value (commonly the column mean or median). Choosing how to handle missing data is itself a modeling decision – never do it silently.

Example Code

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'a': [1, np.nan, 3, 4],
    'b': [np.nan, 2, 3, 4]
})
print(df.isnull())
print(df.isnull().sum())

print(df.dropna()) # drop rows with any NaN
print(df.fillna(df.mean())) # fill NaN with the column mean
```

Exercise

1. Take the Wine DataFrame and artificially set 5 random values in the `magnesium` column to `np.nan` (hint: use `df.loc[idx, 'magnesium'] = np.nan` with random indices).
2. Count how many missing values each column now has.
3. Fill the missing `magnesium` values with the column **median** (not the mean) and confirm there are zero missing values left.

2.5 Feature Engineering: apply & map**Concept**

`apply` runs a custom function over every row or column, and `map` applies a function/dictionary elementwise to a single column. Both are the standard way to create new, derived features – ratios, binned categories, or rule-based flags – before training a model.

Example Code

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
df = pd.DataFrame(iris.data, columns=['sl', 'sw', 'pl', 'pw'])
df['target'] = iris.target

df['petal_area'] = df['pl'] * df['pw'] # new engineered feature
df['sl_category'] = df['sl'].apply(
    lambda x: 'long' if x > 5.8 else 'short'
)
print(df.head(3))
```

Exercise

1. On the Wine DataFrame, create a new feature `phenol_ratio` equal to `flavanoids` divided by `total_phenols`.
2. Using `apply` with a lambda, create a column `high_alcohol` that is 1 if `alcohol > 13` else 0.
3. Use `.map()` with a dictionary to convert the numeric `target` column (0, 1, 2) into the actual class names `wine.target_names`.

2.6 Sorting Values

Concept

`sort_values` orders rows by one or more columns – useful for ranking samples by a score, finding top-*k* outliers, or inspecting the most/least extreme feature values. `sort_index` restores the original row order.

Example Code

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
df = pd.DataFrame(iris.data, columns=['sl', 'sw', 'pl', 'pw'])
df['target'] = iris.target

print(df.sort_values('sl', ascending=False).head(3))
```

Exercise

1. Sort the Wine DataFrame by `alcohol` in descending order and display the top 5 rows.
2. Sort by *two* columns at once: `target` ascending, then `color_intensity` descending (hint: pass lists to `by` and `ascending`).
3. Find the 3 wine samples with the **lowest** hue value (hint: `nsmallest`).

2.7 GroupBy & Aggregation

Concept

`groupby` splits a DataFrame into groups based on a column (e.g. class label), applies an aggregation to each group, and combines the results. This is how you compute *per-class* statistics – essential for understanding whether a feature actually separates your classes before you even train a model.

Example Code

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
df = pd.DataFrame(iris.data, columns=['sl', 'sw', 'pl', 'pw'])
df['target'] = iris.target

print(df.groupby('target').mean())
print(df.groupby('target')['sl'].agg(['mean', 'std', 'count']))
```

Exercise

1. Group the Wine DataFrame by `target` and compute the mean of every feature for each class.
2. For the `alcohol` column, compute the mean, min, and max **per class** in a single `.agg(...)` call.
3. Which class has the highest average `color_intensity`? Answer using code, not by eye.

2.8 Merging & Concatenating DataFrames

Concept

Real datasets often arrive split across multiple files or tables. `pd.merge` joins two DataFrames on a common key (like a SQL JOIN – `how='inner'`, `'left'`, `'right'`, or `'outer'`), and `pd.concat` stacks DataFrames on top of (or beside) each other.

Example Code

```
import pandas as pd

students = pd.DataFrame({'id': [1, 2, 3], 'name': ['Asha', 'Ravi', 'Neel']})
marks = pd.DataFrame({'id': [1, 2, 4], 'marks': [88, 72, 90]})

print(pd.merge(students, marks, on='id', how='inner'))
print(pd.merge(students, marks, on='id', how='left'))

df1 = pd.DataFrame({'x': [1, 2]})
df2 = pd.DataFrame({'x': [3, 4]})
print(pd.concat([df1, df2], ignore_index=True))
```

Exercise

1. Split the Wine DataFrame's first 5 rows into two separate DataFrames: one with columns `alcohol`, `malic_acid` and one with `hue`, `proline`, then re-combine them column-wise using `pd.concat(..., axis=1)`.
2. Create two small DataFrames of 3 rows each with the *same* columns and `pd.concat` them row-wise with `ignore_index=True`.
3. Create a lookup DataFrame mapping `target` (0,1,2) to a `quality_tier` string (e.g. 'A', 'B', 'C'), and merge it onto the first 10 rows of the Wine DataFrame.

2.9 Encoding Categorical Variables

Concept

ML models need numbers, not text. `pd.get_dummies` performs **one-hot encoding** (each category becomes its own 0/1 column, avoiding any false sense of order), while `.map()` with a dictionary performs simple **label encoding** (useful only when categories have a natural order, or for the target column).

Example Code

```
import pandas as pd

df = pd.DataFrame({'city': ['Pune', 'Mumbai', 'Pune', 'Delhi']})
print(pd.get_dummies(df, columns=['city']))

label_map = {'Pune': 0, 'Mumbai': 1, 'Delhi': 2}
df['city_encoded'] = df['city'].map(label_map)
print(df)
```

Exercise

1. Create a small DataFrame with a column `grade` containing `['A', 'B', 'A', 'C', 'B']`. One-hot encode it with `pd.get_dummies`.
2. For the same column, build a label-encoding dictionary (A->2, B->1, C->0, since grades have a natural order) and apply it with `.map()`.
3. One-hot encode the Wine dataset's `target` column (0,1,2) and explain in one sentence (as a code comment) why this is the *correct* encoding for a target with no ordinal relationship.

2.10 Correlation Analysis & Train/Test Sampling

Concept

`.corr()` computes the pairwise correlation between every numeric column – a quick way to spot redundant features (highly correlated with each other) or promising predictors (highly correlated with the target) *before* modeling. `.sample()` randomly shuffles rows, which combined with `.iloc` is one simple way to create a train/test split by hand (in practice you will normally use `sklearn.model_selection.train_test_split`, which does this for you with extra safety checks).

Example Code

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
df = pd.DataFrame(iris.data, columns=['sl', 'sw', 'pl', 'pw'])
print(df.corr())

shuffled = df.sample(frac=1, random_state=1).reset_index(drop=True)
train = shuffled.iloc[:120]
test = shuffled.iloc[120:]
print(train.shape, test.shape)
```

Exercise

1. Compute the correlation matrix of the Wine DataFrame's features. Which two features are most strongly correlated with each other?
2. Using `.sample(frac=1, random_state=0)` followed by `.iloc`, split the Wine DataFrame into a 70% train set and a 30% test set. Print the shape of each.
3. Compute the correlation of every feature with the `target` column and print the single feature with the highest absolute correlation.

3 Capstone Exercise — Putting It All Together

Exercise

Mini-Project: End-to-End Preprocessing of the Wine Dataset. Using *only* NumPy and Pandas (no scikit-learn preprocessing yet), write a single script that performs the following steps in order on `load_wine()`:

1. Load the dataset into a DataFrame and print its shape, dtypes, and summary statistics.
2. Introduce 10 random missing values (`np.nan`) scattered across the feature columns, then report how many missing values exist per column.
3. Impute every missing value with its column **median**.
4. Engineer one new feature of your choice (e.g. a ratio between two existing columns).
5. Standardize every feature column using NumPy (subtract the mean, divide by the standard deviation) so all features are on a comparable scale.
6. One-hot encode the **target** column into three binary columns.
7. Shuffle the rows with a fixed random seed and split the result into an 80% training set and a 20% test set.
8. Print the final shapes of your training and test feature matrices, and confirm (using `.mean()` and `.std()`) that your standardized features have an approximate mean of 0 and standard deviation of 1.

This single exercise mirrors the real preprocessing pipeline you will write for almost every ML project this semester.

Appendix: Quick Reference Cheat Sheet

NumPy

Function	Purpose
<code>np.array(list)</code>	Create an array from a Python list
<code>np.zeros/ones/eye/full</code>	Pre-filled arrays (zeros, ones, identity, constant)
<code>np.arange/linspace</code>	Ranges of evenly spaced numbers
<code>.shape, .ndim, .size, .dtype</code>	Inspect array structure
<code>.reshape(), .ravel(), .T</code>	Change array shape without copying data
<code>X[r, c], X[r1:r2, c1:c2]</code>	Index / slice rows and columns
<code>X[X > k]</code>	Boolean masking (conditional selection)
<code>+, -, *, / (broadcasting)</code>	Elementwise arithmetic across mismatched shapes
<code>.sum/mean/std/var(axis=)</code>	Aggregations along rows (1) or columns (0)
<code>.argmin()/ .argmax()</code>	Index of the minimum / maximum value
<code>np.vstack/hstack/concatenate</code>	Combine arrays
<code>np.split()</code>	Divide an array into equal parts
<code>X.dot(w), X @ w</code>	Matrix / vector product
<code>np.linalg.inv/det/norm</code>	Matrix inverse, determinant, vector norm
<code>np.random.seed()</code>	Fix the random seed for reproducibility
<code>np.random.rand/randn/randint</code>	Random numbers: uniform, normal, integer
<code>np.random.choice/shuffle</code>	Random sampling and shuffling
<code>np.sort/argsort</code>	Sort values / get sorting indices
<code>np.unique(a, return_counts=True)</code>	Distinct values and their counts
<code>np.where(cond, a, b)</code>	Elementwise conditional selection
<code>np.clip(a, lo, hi)</code>	Bound values to a range

Pandas

Function	Purpose
<code>pd.Series(), pd.DataFrame()</code>	Create a labeled 1-D / 2-D structure
<code>pd.DataFrame(sklearn_data.data, ...)</code>	Load a scikit-learn dataset into a DataFrame
<code>.head()/ .tail()</code>	Preview first / last rows
<code>.shape, .info(), .describe()</code>	Structure, types & missing values, summary stats
<code>df['col'], df[['c1', 'c2']]</code>	Select one / multiple columns
<code>.loc[], .iloc[]</code>	Select by label / by integer position
<code>df[df['col'] > x]</code>	Boolean filtering of rows
<code>.isnull().sum()</code>	Count missing values per column
<code>.dropna(), .fillna()</code>	Remove or impute missing values

Function	Purpose
<code>.apply()</code> , <code>.map()</code>	Apply a function row/column-wise or elementwise
<code>.sort_values()</code> , <code>.sort_index()</code>	Sort rows by value(s) / by index
<code>.groupby(col).agg()</code>	Per-group aggregation (mean, std, count, ...)
<code>pd.merge(df1, df2, on=, how=)</code>	SQL-style join of two DataFrames
<code>pd.concat([df1, df2])</code>	Stack DataFrames vertically / horizontally
<code>pd.get_dummies()</code>	One-hot encode a categorical column
<code>.map(dict)</code>	Label-encode a categorical column
<code>.corr()</code>	Pairwise correlation between numeric columns
<code>.sample(frac=1, random_state=)</code>	Shuffle rows (with reproducibility)

End of Practical Session 1 Lab Sheet.